

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192472208
Name	Venganapalli Sireesha

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I V.Sireesha(192472208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V.Sireesha

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Q1. End-to-End System Design

Real-Time Social Media Analysis and Recommendation System

1. Problem Understanding and Requirements

The proposed system analyzes large amounts of real-time social media data and provides intelligent recommendations to users.

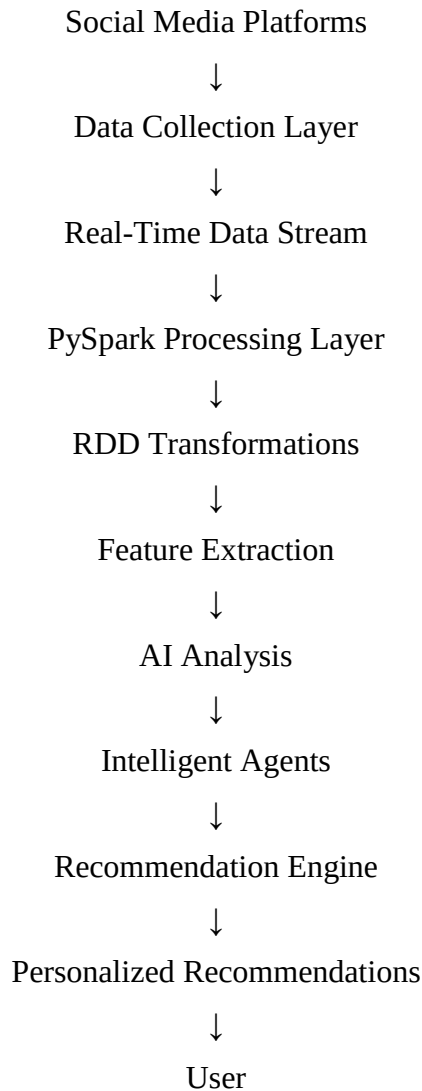
Objectives:

- Collect real-time social media posts.
- Process large-scale data using PySpark.
- Remove unwanted and irrelevant data.
- Identify user interests, topics and sentiments.
- Generate personalized recommendations.

Requirements:

- Real-time data processing.
- Distributed processing using PySpark.
- Fault-tolerant data processing.
- Intelligent agents for analysis and recommendation.
- Scalable architecture for increasing data.

2. System Architecture



The architecture is divided into data collection, processing, AI analysis and recommendation layers. This provides a structured and scalable design, as required by the architecture rubric.

3. PySpark and RDD Operations

The incoming social media data is converted into RDDs.

- **map()** – converts posts into structured records.
- **filter()** – removes unwanted or invalid posts.
- **flatMap()** – extracts words and hashtags.
- **reduceByKey()** – calculates the frequency of words or topics.
- **groupByKey()** – groups posts based on users or topics.
- **count()** – counts the number of posts.
- **collect()** – obtains the required results.

These transformations and actions provide distributed processing of large datasets. The rubric specifically expects correct and efficient RDD transformations/actions.

4. Intelligent Agents

Data Collection Agent

- Collects social media data.
- Sends data to the processing layer.

Analysis Agent

- Analyzes topics and sentiments.
- Identifies user interests.

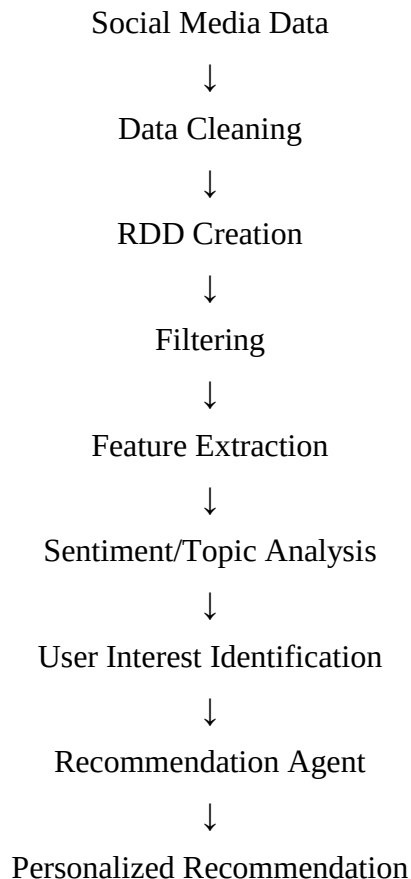
Recommendation Agent

- Uses analysis results to generate personalized recommendations.

Monitoring Agent

- Monitors processing and detects failures.

5. Data Flow



6. Scalability and Fault Tolerance

- Data is divided into RDD partitions.
- Multiple worker nodes process partitions in parallel.
- More nodes can be added when data increases.
- RDD lineage allows lost partitions to be recomputed.
- Distributed processing reduces the load on a single machine.

7. Innovation

The system can combine user interests, trending topics, hashtags and sentiment to provide more relevant recommendations.

8. Result

The proposed system provides real-time, personalized and scalable recommendations from large-scale social media data.

Q2. Distributed Intelligent Agent System

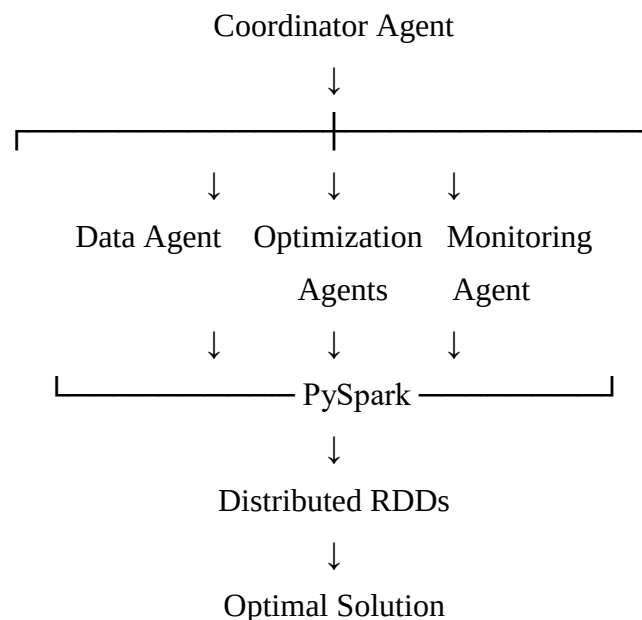
Multi-Agent System for Large-Scale Optimization

1. Problem Understanding

Large-scale optimization problems contain huge amounts of data and many possible solutions. A single agent may take more time to find an optimal solution.

The proposed system uses **multiple intelligent agents and PySpark** to divide the problem into smaller tasks and solve them in parallel.

2. System Architecture



3. Agent Roles

Coordinator Agent

- Divides the optimization problem.
- Assigns tasks to agents.
- Combines the results.

Data Processing Agent

- Cleans and prepares input data.
- Creates RDDs.

Optimization Agents

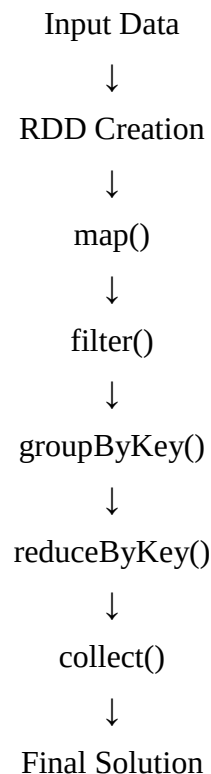
- Solve different parts of the problem.
- Generate possible solutions.

Monitoring Agent

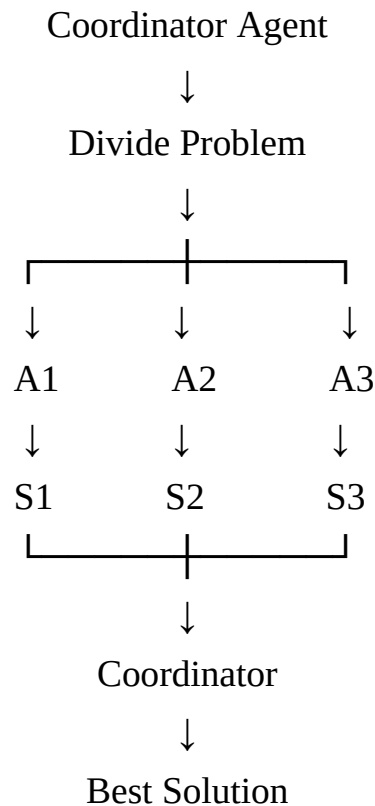
- Monitors tasks.
- Detects failures.

This gives clear agent roles and logic, which directly addresses the Excellent intelligent-agent criterion.

4. PySpark RDD Operations



5. Agent Collaboration



6. Scalability and Fault Tolerance

- Multiple agents work simultaneously.
- RDD partitions are processed in parallel.
- Additional worker nodes can be added.
- Failed computations can be recomputed.
- Large datasets can be distributed across the cluster.

7. Innovation

The coordinator can dynamically assign more tasks to available agents when the workload changes.

8. Result

The proposed multi-agent system solves large optimization problems efficiently through parallel processing, agent collaboration and distributed computing.

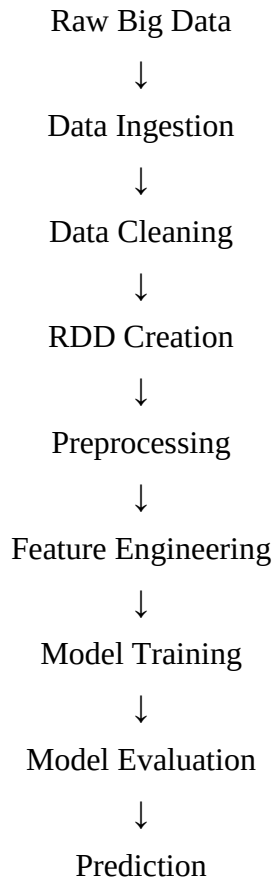
Q3. Big Data AI Pipeline Design

Distributed AI Pipeline Using PySpark RDD

1. Problem Understanding

Large datasets require efficient preprocessing, feature extraction and model training. A distributed PySpark pipeline can process the data across multiple machines.

2. Complete Pipeline

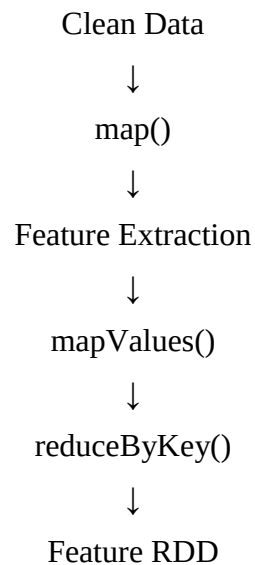


3. Data Preprocessing

RDD operations are used to clean and prepare the data.

- `filter()` – removes invalid records.
- `map()` – transforms the data.
- `flatMap()` – separates data into smaller elements.

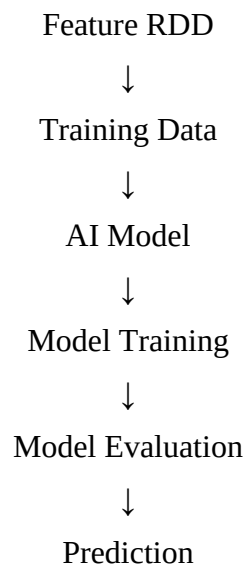
4. Feature Engineering



Features are extracted from the processed data and prepared for model training.

5. Model Training

The engineered features are provided to an AI model.



6. Fault Tolerance

RDDs maintain lineage information. If a partition is lost, PySpark can recompute that partition from the previous transformations.

7. Scalability

- Data is divided into partitions.
- Multiple machines process partitions simultaneously.
- Additional workers can be added for large datasets.
- Distributed processing reduces processing time.

These points address the rubric's requirement for scalability, fault tolerance and parallelism.

8. Intelligent Agents

Preprocessing Agent – cleans data.

Feature Engineering Agent – extracts useful features.

Training Agent – trains the AI model.

Evaluation Agent – evaluates the trained model.

9. Result

The proposed pipeline provides a scalable, distributed and fault-tolerant AI processing system.

Q4. Real-Time Decision-Making System

Smart Traffic Management Using PySpark and Intelligent Agents

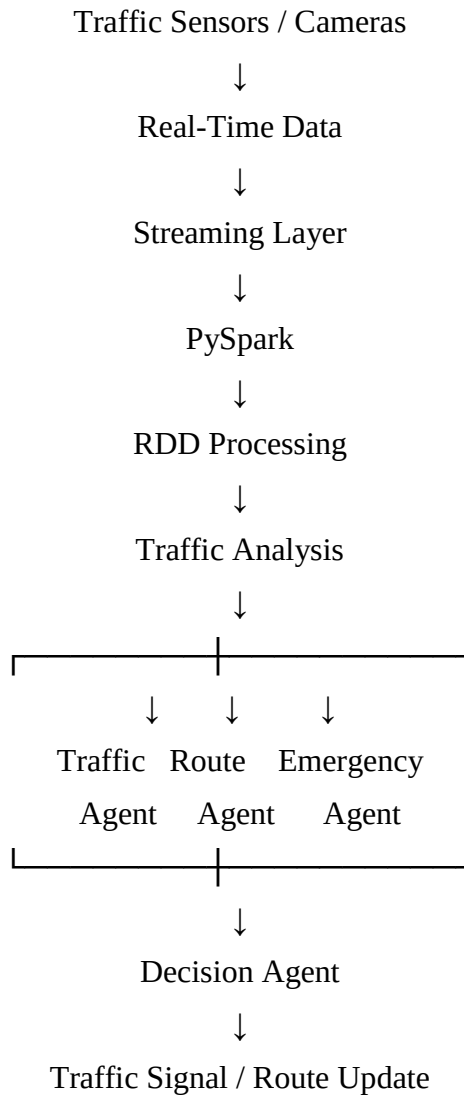
1. Problem Understanding

Modern cities generate traffic data continuously through sensors, cameras and GPS devices. The system analyzes this data in real time and makes intelligent traffic decisions.

Objectives:

- Monitor traffic continuously.
- Identify congested roads.
- Find alternative routes.
- Give priority to emergency vehicles.
- Make real-time traffic decisions.

2. System Architecture



3. RDD Processing

- **filter()** – removes incorrect sensor data.
- **map()** – converts sensor readings into useful records.
- **reduceByKey()** – calculates traffic density.
- **sortBy()** – identifies highly congested roads.

4. Intelligent Agents

Traffic Monitoring Agent

- Monitors traffic density.

Route Optimization Agent

- Finds less congested routes.

Emergency Agent

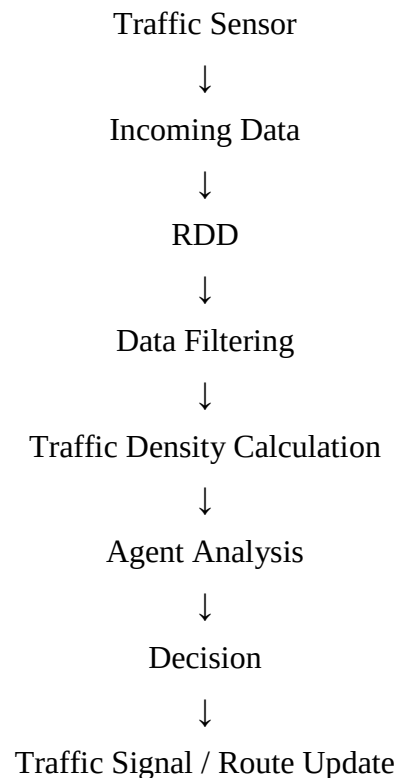
- Gives priority to emergency vehicles.

Decision Agent

- Makes final traffic control decisions.

The agents collaborate to convert processed data into real-time decisions, satisfying the strong agent-integration requirement.

5. Data Flow



6. Scalability and Fault Tolerance

- Traffic data is distributed across Spark workers.
- Multiple sensors can be processed simultaneously.
- New nodes can be added when traffic data increases.
- Failed RDD partitions can be recomputed.
- Parallel processing supports real-time decisions.

7. Innovation

The system can use historical traffic patterns along with current traffic data to predict congestion before it becomes severe.

8. Result

The system provides fast and intelligent traffic management, helping reduce congestion and improve emergency response.

Q5. Cloud-Based Distributed AI Application

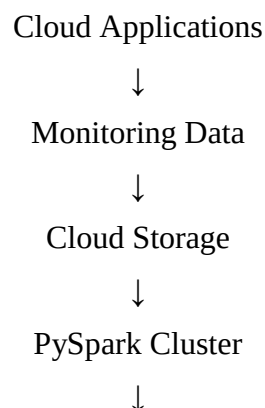
Cloud-Based AI System for Dynamic Resource Allocation

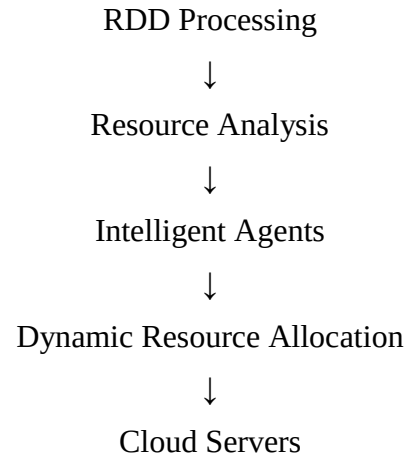
1. Problem Understanding

Cloud applications continuously generate workloads and require CPU, memory and storage resources. Static allocation may cause resource wastage or system overload.

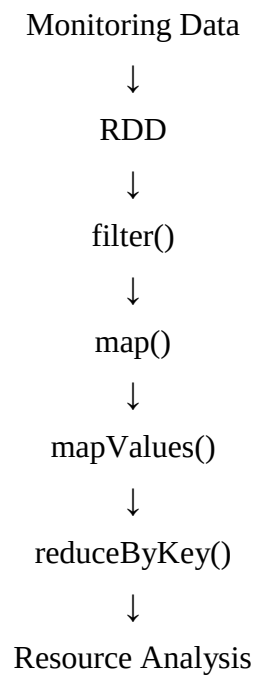
The proposed system uses PySpark and intelligent agents to analyze cloud data and dynamically allocate resources.

2. System Architecture





3. PySpark and RDD Operations



4. Intelligent Agents

Monitoring Agent

- Collects CPU, memory and network usage.

Prediction Agent

- Predicts future resource requirements.

Allocation Agent

- Allocates resources according to workload.

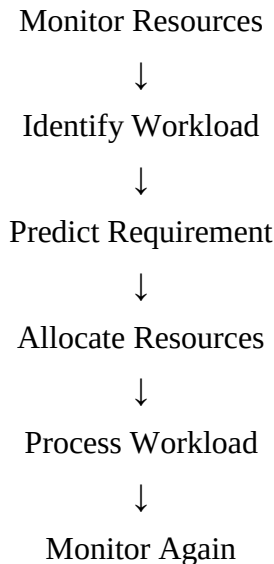
Learning Agent

- Learns from previous resource usage.

Fault Detection Agent

- Identifies overloaded or failed resources.

5. Dynamic Resource Allocation



6. Adaptive Learning

The learning agent studies previous resource usage and identifies:

- Peak usage periods.
- Frequently overloaded servers.
- Resource usage patterns.
- Future resource requirements.

The system uses these patterns to improve future resource allocation.

7. Scalability and Fault Tolerance

- PySpark distributes processing across cloud nodes.
- More nodes can be added when workload increases.
- Resources can be dynamically increased or decreased.
- RDD lineage supports recovery from lost partitions.
- Workloads can be shifted to available resources.

8. Innovation

The system combines real-time monitoring, prediction, intelligent agents and adaptive learning to automatically optimize cloud resources.

9. Result

The proposed system provides large-scale analytics, dynamic resource allocation, scalability, fault tolerance and adaptive AI-based cloud management.